Experiment No. 7

Experiment Name - Customer Segmentation using K-Means Clustering

Steps

NAME : Atul Krishna

COURSE : B.Tech CSE

SECTION - B

SUBJECT : Mastery in Data Analytics and Visualizations and Career Advancement

ROLL NO. : 14

CU ID : CU24250231

1. Select a suitable dataset such as the Mall Customer Segmentation Dataset, Online Retail Dataset, or Wholesale Customers Dataset.

```
# Selecting a Real-World Dataset
from google.colab import files
uploaded = files.upload()

import os
print(os.listdir())

dataset_name = "Online Retail Dataset"

print("Selected Dataset:", dataset_name)
print("Dataset selected successfully!")
```

```
Choose Files Online Retail Dataset.csv
Online Retail Dataset.csv(text/csv) - 48581636 bytes, last modified: 9/14/2026 - 100% done
Saving Online Retail Dataset.csv to Online Retail Dataset (1).csv
['.config', 'Online Retail Dataset.csv', 'Online Retail Dataset (1).csv', 'sample_data']
Selected Dataset: Online Retail Dataset
Dataset selected successfully!
```

2. Import the required Python libraries and load the dataset into a Pandas DataFrame.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load the dataset
df = pd.read_csv("Online Retail Dataset.csv", encoding="latin1")

# Display confirmation message
print("Dataset loaded successfully!")

# Display the shape of the dataset
print("Shape of dataset:", df.shape)

# Display the first five rows
print("\nFirst five rows of the dataset:")
print(df.head())
```

```
Dataset loaded successfully!
Shape of dataset: (541909, 8)

First five rows of the dataset:
  InvoiceNo StockCode                          Description  Quantity  \
0    536365    85123A   WHITE HANGING HEART T-LIGHT HOLDER         6
1    536365     71053                  WHITE METAL LANTERN         6
2    536365    84406B       CREAM CUPID HEARTS COAT HANGER         8
3    536365    84029G  KNITTED UNION FLAG HOT WATER BOTTLE         6
4    536365    84029E       RED WOOLLY HOTTIE WHITE HEART.         6

           InvoiceDate  UnitPrice  CustomerID         Country
0  2010-12-01 08:26:00       2.55     17850.0  United Kingdom
1  2010-12-01 08:26:00       3.39     17850.0  United Kingdom
2  2010-12-01 08:26:00       2.75     17850.0  United Kingdom
3  2010-12-01 08:26:00       3.39     17850.0  United Kingdom
4  2010-12-01 08:26:00       3.39     17850.0  United Kingdom
```

3. Perform data preprocessing by handling missing values, removing duplicates, and selecting relevant features such as Age, Annual Income, and Spending Score.

```
# Display missing values before preprocessing
print("Missing values before preprocessing:")
print(df.isnull().sum())

# Remove rows with missing CustomerID
df = df.dropna(subset=["CustomerID"])

# Remove duplicate rows
df = df.drop_duplicates()

# Convert InvoiceDate to datetime format
df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"])

# Remove cancelled transactions
df = df[~df["InvoiceNo"].astype(str).str.startswith("C")]

# Remove invalid transactions
df = df[df["Quantity"] > 0]
df = df[df["UnitPrice"] > 0]

# Select relevant features for customer segmentation
selected_features = df[
    ["CustomerID", "InvoiceDate", "InvoiceNo", "Quantity", "UnitPrice"]
]

# Display preprocessing results
print("\nData preprocessing completed successfully!")

print("\nShape after preprocessing:", df.shape)

print("\nSelected Features:")
print(selected_features.head())
```

```
Missing values before preprocessing:
InvoiceNo           0
StockCode           0
Description      1454
Quantity            0
InvoiceDate         0
UnitPrice           0
CustomerID     135080
Country             0
dtype: int64

Data preprocessing completed successfully!

Shape after preprocessing: (392692, 8)

Selected Features:
   CustomerID         InvoiceDate InvoiceNo  Quantity  UnitPrice
0     17850.0 2010-12-01 08:26:00    536365         6       2.55
1     17850.0 2010-12-01 08:26:00    536365         6       3.39
2     17850.0 2010-12-01 08:26:00    536365         8       2.75
3     17850.0 2010-12-01 08:26:00    536365         6       3.39
4     17850.0 2010-12-01 08:26:00    536365         6       3.39
```

4. Normalize or standardize the selected features to ensure that all variables contribute equally during clustering.

```
# Calculate total purchase amount
df["TotalAmount"] = df["Quantity"] * df["UnitPrice"]

# Create customer-level RFM features
reference_date = df["InvoiceDate"].max() + pd.Timedelta(days=1)

rfm = df.groupby("CustomerID").agg({
    "InvoiceDate": lambda x: (reference_date - x.max()).days,
    "InvoiceNo": "nunique",
    "TotalAmount": "sum"
})

rfm.columns = ["Recency", "Frequency", "Monetary"]

# Select features for clustering
features = rfm[["Recency", "Frequency", "Monetary"]]

# Standardize the features
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaled_features = scaler.fit_transform(features)

print("Selected features standardized successfully!")
print(scaled_features[:5])
```

```
Selected features standardized successfully!
[[ 2.33457414 -0.4250965   8.36301037]
 [-0.90534032  0.3544168   0.2516989 ]
 [-0.17535959 -0.03533985 -0.02798783]
```

```
[-0.73534481 -0.4250965  -0.03240559]
 [ 2.17457836 -0.4250965  -0.19081155]]
```

5. Determine the optimal number of clusters using the Elbow Method and validate the result using the Silhouette Score.

```
# Determining Optimal Number of Clusters
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Elbow Method
inertia = []

for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(scaled_features)
    inertia.append(kmeans.inertia_)

# Plot Elbow Curve
plt.figure(figsize=(6, 3))
plt.plot(range(2, 11), inertia, marker="o")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.xticks(range(2, 11))
plt.grid(True)
plt.show()

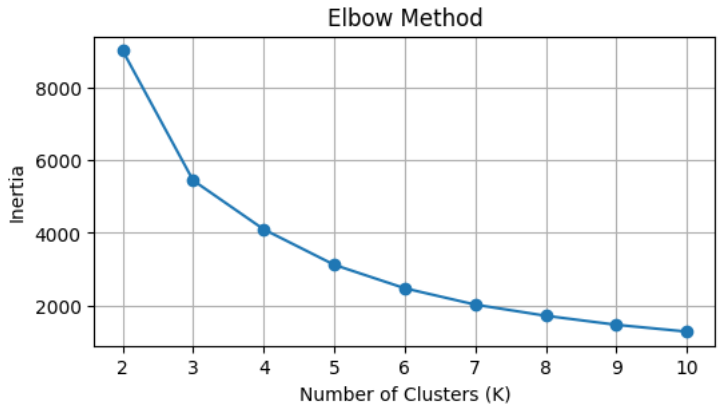
# Silhouette Score
silhouette_scores = []

for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(scaled_features)
    score = silhouette_score(scaled_features, labels)
    silhouette_scores.append(score)

# Display Silhouette Scores
for k, score in zip(range(2, 11), silhouette_scores):
    print("K =", k, "Silhouette Score =", round(score, 4))

# Find the K with the highest Silhouette Score
best_k = range(2, 11)[np.argmax(silhouette_scores)]

print("\nOptimal Number of Clusters:", best_k)
print("Highest Silhouette Score:", round(max(silhouette_scores), 4))
```




```
K = 2 Silhouette Score = 0.8958
K = 3 Silhouette Score = 0.5942
K = 4 Silhouette Score = 0.6162
K = 5 Silhouette Score = 0.6165
K = 6 Silhouette Score = 0.5983
K = 7 Silhouette Score = 0.5165
K = 8 Silhouette Score = 0.4859
K = 9 Silhouette Score = 0.4784
K = 10 Silhouette Score = 0.4792

Optimal Number of Clusters: 2
Highest Silhouette Score: 0.8958
```

6. Apply the K-Means Clustering algorithm with the selected value of K to group customers into distinct clusters.

```
# Apply K-Means using the selected optimal K
kmeans = KMeans(
    n_clusters=best_k,
    random_state=42,
    n_init=10
)

# Assign clusters to customers
rfm["Cluster"] = kmeans.fit_predict(scaled_features)
```

```
print("K-Means clustering applied successfully!")
print("Number of clusters:", best_k)

# Display the first few customers with their cluster
print("\nCustomer clusters:")
print(rfm.head())

# Display number of customers in each cluster
print("\nNumber of customers in each cluster:")
print(rfm["Cluster"].value_counts().sort_index())
```

```
K-Means clustering applied successfully!
Number of clusters: 2

Customer clusters:
            Recency  Frequency  Monetary  Cluster
CustomerID
12346.0         326          1  77183.60        0
12347.0           2          7   4310.00        0
12348.0          75          4   1797.24        0
12349.0          19          1   1757.55        0
12350.0         310          1    334.40        0

Number of customers in each cluster:
Cluster
0    4312
1      26
Name: count, dtype: int64
```

7. Visualize the clusters using scatter plots and highlight the cluster centroids for better interpretation.

```
# Calculate cluster centroids
centroids = kmeans.cluster_centers_

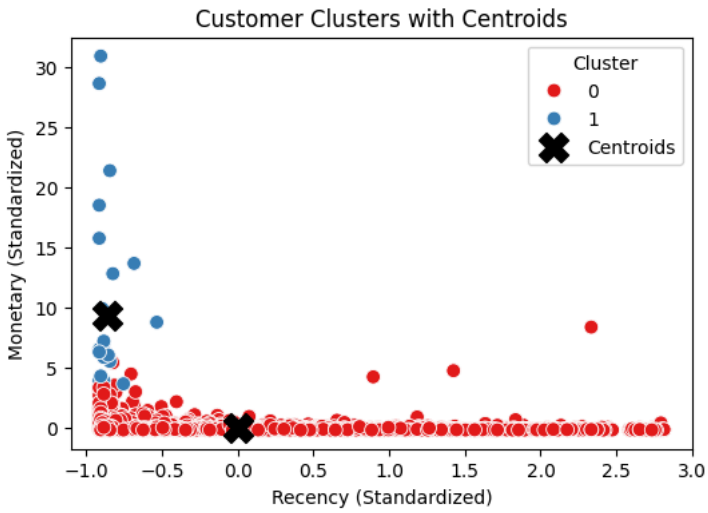
# Create scatter plot
plt.figure(figsize=(6, 4))

sns.scatterplot(
    x=scaled_features[:, 0],
    y=scaled_features[:, 2],
    hue=rfm["Cluster"],
    palette="Set1",
    s=60
)

# Highlight cluster centroids
plt.scatter(
    centroids[:, 0],
    centroids[:, 2],
    marker="X",
    s=250,
    c="black",
    label="Centroids"
)

plt.xlabel("Recency (Standardized)")
plt.ylabel("Monetary (Standardized)")
plt.title("Customer Clusters with Centroids")
plt.legend(title="Cluster")

plt.show()
```



8. Analyze the characteristics of each customer segment based on income, spending behavior, or purchasing patterns.

```
# Analyzing the Characteristics of Each Customer Segment
cluster_summary = rfm.groupby("Cluster").agg(
    CustomerCount=("Cluster", "size"),
    AverageRecency=("Recency", "mean"),
    AverageFrequency=("Frequency", "mean"),
    AverageMonetary=("Monetary", "mean")
).round(2)

print("Customer Segment Characteristics:")
print(cluster_summary)
```

```
Customer Segment Characteristics:
         CustomerCount  AverageRecency  AverageFrequency  AverageMonetary
Cluster
0                 4312           93.06              3.90          1543.54
1                   26            6.04             66.42         85826.08
```

9. Compare the identified customer groups and suggest suitable marketing strategies for each segment.

- The customers are divided into different segments based on their Recency, Frequency, and Monetary values.
- Customers with low Recency and high Monetary value are active and valuable customers. They can be targeted with loyalty rewards, exclusive offers, and premium products.
- Customers with high Recency but high Monetary value have made valuable purchases in the past but are currently less active. Personalized discounts, reminders, and re-engagement campaigns can be used to bring them back.
- Customers with high Frequency show regular purchasing behavior. Membership benefits, repeat-purchase offers, and special discounts can encourage them to continue purchasing.
- Customers with lower Frequency and Monetary value can be targeted with introductory discounts, promotional offers, and product recommendations to increase their engagement.

Thus, different customer segments require different marketing strategies based on their purchasing behavior.

10. Summarize the clustering results and discuss how customer segmentation can improve business decision-making.

Clustering Results:

- The K-Means clustering algorithm was used to divide customers into different groups based on their purchasing behavior. The customers were segmented using Recency, Frequency, and Monetary (RFM) features.
- The identified clusters represent customers with similar purchasing patterns and spending behavior. Some groups may contain frequent and high-value customers, while others may include less active or low-value customers.

Benefits of Customer Segmentation:

- Helps identify valuable and loyal customers.
- Helps understand different purchasing patterns.
- Allows businesses to create targeted marketing campaigns.
- Helps provide personalized offers and discounts.
- Improves customer retention and engagement.
- Helps businesses allocate marketing resources more effectively.

Conclusion:

- Customer segmentation using K-Means clustering helps businesses understand their customers more effectively. By analyzing different customer groups, businesses can make better decisions related to marketing, promotions, customer retention, and sales strategies. Thus, customer segmentation can improve customer satisfaction and support better business decision-making.

QUESTIONS AND ANSWERS

1. What is clustering? How does it differ from classification?

Clustering is an unsupervised machine learning technique used to group similar data points into clusters based on their characteristics.

Classification is a supervised machine learning technique where data is assigned to predefined categories or classes using labeled training data.

Difference:

- Clustering works with unlabeled data.
- Classification works with labeled data.
- Clustering discovers hidden groups in data.
- Classification predicts the class of new data.

2. Explain the working principle of the K-Means Clustering algorithm.

K-Means is a clustering algorithm that divides data into K different clusters.

The basic steps are:

- Choose the number of clusters (K).
- Randomly initialize K cluster centroids.

- Assign each data point to the nearest centroid.
- Calculate new centroids for the clusters.
- Repeat the assignment and centroid calculation steps until the centroids become stable.

The final result is K clusters containing data points with similar characteristics.

3. Why is feature scaling important before applying K-Means clustering?

Feature scaling is important because K-Means uses distance calculations to assign data points to clusters.

If different features have different scales, a feature with larger numerical values may have more influence on the clustering process.

Scaling puts all features on a similar scale so that each feature contributes more equally to the distance calculation.

Common scaling techniques include Standardization and Normalization.

4. What is the Elbow Method, and how does it help determine the optimal number of clusters?

The Elbow Method is a technique used to determine a suitable number of clusters for K-Means.

It calculates the Within-Cluster Sum of Squares (WCSS), also called inertia, for different values of K.

The values of K are plotted against their corresponding inertia values. The point where the decrease in inertia starts becoming less significant is called the "elbow point".

The K value at the elbow is generally selected as a suitable number of clusters.

5. What is the Silhouette Score? How is it used to evaluate clustering performance?

The Silhouette Score is a metric used to measure the quality of clustering.

It measures how similar a data point is to its own cluster compared to other clusters.

The score ranges from -1 to 1:

- A value close to 1 indicates well-separated clusters.
- A value close to 0 indicates overlapping clusters.
- A negative value may indicate that data points are assigned to inappropriate clusters.

A higher Silhouette Score generally indicates better-defined clusters.

6. Why is K-Means considered an unsupervised machine learning algorithm?

K-Means is considered an unsupervised machine learning algorithm because it does not require labeled output data.

The algorithm works only with the input features and automatically identifies groups of similar data points.

The clusters are discovered from the structure and similarity present in the data without predefined class labels.

7. Mention any four real-world applications of customer segmentation.

Four real-world applications of customer segmentation are:

- Targeted Marketing – Businesses can create marketing campaigns for specific customer groups.
- Personalized Offers – Customers can receive discounts and offers based on their purchasing behavior.
- Customer Retention – Businesses can identify inactive customers and use special offers to encourage them to return.
- Product Recommendation – Businesses can recommend products based on customers' previous purchases and preferences.

8. What are the limitations of the K-Means algorithm?

The main limitations of K-Means are:

- The number of clusters (K) must be selected before applying the algorithm.
- The algorithm is sensitive to the initial selection of centroids.
- It can be affected by outliers.
- It works best when clusters are relatively spherical and similar in size.
- It may not perform well when clusters have irregular shapes.
- Results can change depending on the initial centroid positions.

9. How can businesses use customer segmentation to improve marketing and customer retention?

Businesses can use customer segmentation to divide customers into groups based on their purchasing behavior, preferences, and value.

This helps businesses:

- Create targeted marketing campaigns.
- Provide personalized discounts and offers.
- Identify valuable and loyal customers.
- Re-engage inactive customers.
- Recommend relevant products.
- Improve customer satisfaction and retention.

Therefore, customer segmentation helps businesses provide more personalized services and build stronger relationships with customers.

10. Compare K-Means Clustering with Hierarchical Clustering based on their working principles and applications.

K-Means Clustering:

- K-Means divides data into a predefined number of clusters.

- It uses centroids to represent clusters.
- Data points are assigned to the nearest centroid.
- It is generally faster for large datasets.
- It is commonly used for customer segmentation, market analysis, and grouping similar data.

Hierarchical Clustering:

- Hierarchical clustering creates a hierarchy of clusters.
- It can start with individual data points and progressively merge them into larger clusters.
- The result can be represented using a dendrogram.
- The number of clusters can be selected after observing the hierarchy.
- It is useful for smaller datasets, customer grouping, biological analysis, and data exploration.

Main Difference:

- K-Means directly creates a specified number of clusters using centroids, whereas Hierarchical Clustering builds a hierarchy of clusters and represents their relationships using a dendrogram.